

Guía de Ejercicios de Práctica – Parcial 2 (SQL Server)

Instrucciones:

- Todos los ejercicios deben resolverse usando SQL Server.
- No se permite el uso de herramientas externas (Copilot, ChatGPT, etc.).
- Agregá comentarios en tu script explicando cada paso (-- Comentario aquí).
- Guardá y entregá el archivo con el nombre: Apellido_Nombre_ParcialPractica.sql

Ejercicio 1 – JOIN con condición lógica (Ejemplo resuelto)

Mostrá los nombres de los productos cuya cantidad total vendida en el último mes supera las 20 unidades. Incluí también la categoría del producto. Usá al menos un **JOIN** y una función de sistema como GETDATE() o DATEADD().

```
SELECT
    P.NombreProducto,
    C.NombreCategoria,
    SUM(V.Cantidad) AS TotalVendida
FROM Ventas V
INNER JOIN Productos P ON V.ID_Producto = P.ID_Producto
INNER JOIN Categorias C ON P.ID_Categoria = C.ID_Categoria
WHERE V.FechaVenta >= DATEADD(MONTH, -1, GETDATE())
GROUP BY P.NombreProducto, C.NombreCategoria
HAVING SUM(V.Cantidad) > 20;
```

Ejercicio 2 – Subconsulta anidada y funciones agregadas

Listá los clientes cuyo total acumulado de compras sea **mayor al promedio de todos los totales de compra**. Mostrá: nombre del cliente, total comprado. Usá una **subconsulta anidada** y una función de agregación.

```
SELECT
    C.NombreCliente,
    SUM(P.ImporteTotal) AS TotalComprado
FROM Clientes C
INNER JOIN Pedidos P ON C.ID_Cliente = P.ID_Cliente
GROUP BY C.NombreCliente
HAVING SUM(P.ImporteTotal) > (
    SELECT AVG(TotalPorCliente)
    FROM (
        SELECT ID_Cliente, SUM(ImporteTotal) AS TotalPorCliente
        FROM Pedidos
        GROUP BY ID_Cliente
    ) AS Totales
);
```

Ejercicio 3 – Procedimiento almacenado con validación

Creá un procedimiento almacenado llamado sp_RegistrarNuevoPedido que reciba el ID de un cliente, la fecha del pedido y el importe. El procedimiento debe:

- Verificar si el cliente existe.
- Si no existe, mostrar un mensaje de error y cancelar.

- Si existe, insertar el pedido en la tabla correspondiente.

```
CREATE OR ALTER PROCEDURE sp_RegistrarNuevoPedido
    @ID_Cliente INT,
    @FechaPedido DATE,
    @ImporteTotal DECIMAL(10, 2)
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM Clientes WHERE ID_Cliente = @ID_Cliente)
    BEGIN
        RAISERROR('El cliente no existe.', 16, 1);
        RETURN;
    END;

    INSERT INTO Pedidos (ID_Cliente, FechaPedido, ImporteTotal)
    VALUES (@ID_Cliente, @FechaPedido, @ImporteTotal);

    PRINT 'Pedido registrado correctamente.';
END;
```

Ejercicio 4 – Función escalar con lógica condicional

Creá una función escalar llamada fn_CalcularCategoriaCliente que reciba el total de compras del cliente y retorne una categoría:

- 'Bronce' si el total es menor a 10.000
- 'Plata' si está entre 10.000 y 50.000
- 'Oro' si supera los 50.000

Usá CASE o lógica condicional.

```
CREATE OR ALTER FUNCTION fn_CalcularCategoriaCliente(@TotalCompras DECIMAL(10, 2))
RETURNS VARCHAR(20)
AS
BEGIN
    DECLARE @Categoria VARCHAR(20);

    SET @Categoria =
        CASE
            WHEN @TotalCompras < 10000 THEN 'Bronce'
            WHEN @TotalCompras BETWEEN 10000 AND 50000 THEN 'Plata'
            ELSE 'Oro'
        END;

    RETURN @Categoria;
END;
```

Ejercicio 5 – Creación de vista con filtros

Creá una vista llamada vw_ClientesActivos2024 que muestre el nombre y el email de los clientes que realizaron al menos un pedido durante 2024. Usá YEAR() para filtrar.

```
CREATE OR ALTER VIEW vw_ClientesActivos2024
AS
SELECT DISTINCT
    C.NombreCliente,
    C.Email
FROM Clientes C
INNER JOIN Pedidos P ON C.ID_Cliente = P.ID_Cliente
```

```
WHERE YEAR(P.FechaPedido) = 2024;
```